

EduSync

AI-Powered Educational Curriculum Design

Project Description

EduSync is a **Generative AI-powered educational platform** developed to revolutionize the process of academic curriculum design and personalized learning. Traditional curriculum development is a time-consuming and manual process that requires significant effort from educators to create structured syllabi, learning outcomes, assessments, and learning resources. EduSync solves this challenge by leveraging **Generative Artificial Intelligence** through the **Groq API** and the **Llama 3.3 70B language model** to automatically generate high-quality, structured, and industry-relevant educational content.

The platform is designed for two major users: **educators and students**. Educators can provide course details such as course name, academic level, number of semesters, weekly hours, total credits, and technology specialization. Based on these inputs, the AI generates a complete curriculum that includes semester-wise syllabus, Bloom's Taxonomy-based learning outcomes, assignments, quizzes, examinations, mini projects, capstone projects, skill mapping, and recommended learning resources.

Students can utilize EduSync to create personalized weekly study plans, receive AI-based resource recommendations, access customized assessments, and track their learning progress through analytics dashboards.

EduSync is built using a modern full-stack architecture consisting of a responsive frontend developed with **HTML5, CSS3, JavaScript, and Bootstrap**, a robust backend using **Python FastAPI**, an **SQLite database** for storing user and curriculum data, **JWT-based authentication** for secure user management, and a **PDF generation module** for downloading curriculum reports.

By combining AI-driven content generation with modern web technologies, EduSync significantly reduces the time and effort required for curriculum planning, enhances educational quality, supports personalized learning, and provides a scalable solution for the future of intelligent education systems.

Scenarios

Scenario 1: Educator Creates AI-Generated Curriculum

An educator logs into EduSync and enters course details such as course name, educational level, number of semesters, weekly hours, credits, and specialization. The system sends this information to the Groq API powered by the Llama 3.3 70B model, which generates a complete curriculum containing semester-wise syllabus, Bloom's Taxonomy-based learning outcomes, assessments, projects, and recommended resources. The educator can review, edit, and export the curriculum as a PDF.

Scenario 2: Student Generates Personalized Learning Plan

A student registers on EduSync and provides learning preferences and educational level. The AI analyzes the information and creates a personalized weekly study plan, recommends relevant learning resources, and generates practice quizzes and assessments. Students can follow the structured roadmap and monitor their progress through the analytics dashboard.

Scenario 3: Python for Data Science Program

An online platform creates a 1-year Python for Data Science program by providing inputs like Python skill, BTech level, 2 semesters, and data science industry focus. The AI model generates a balanced curriculum progressing from Python fundamentals to advanced data science applications. Courses include NumPy, Pandas, statistics, web scraping, and visualization with Matplotlib and Seaborn. Practical projects reinforce real-world data science skills. A professional PDF syllabus is produced to support student recruitment and promotions.

Scenario 4: Curriculum Analysis and Skill Gap Identification

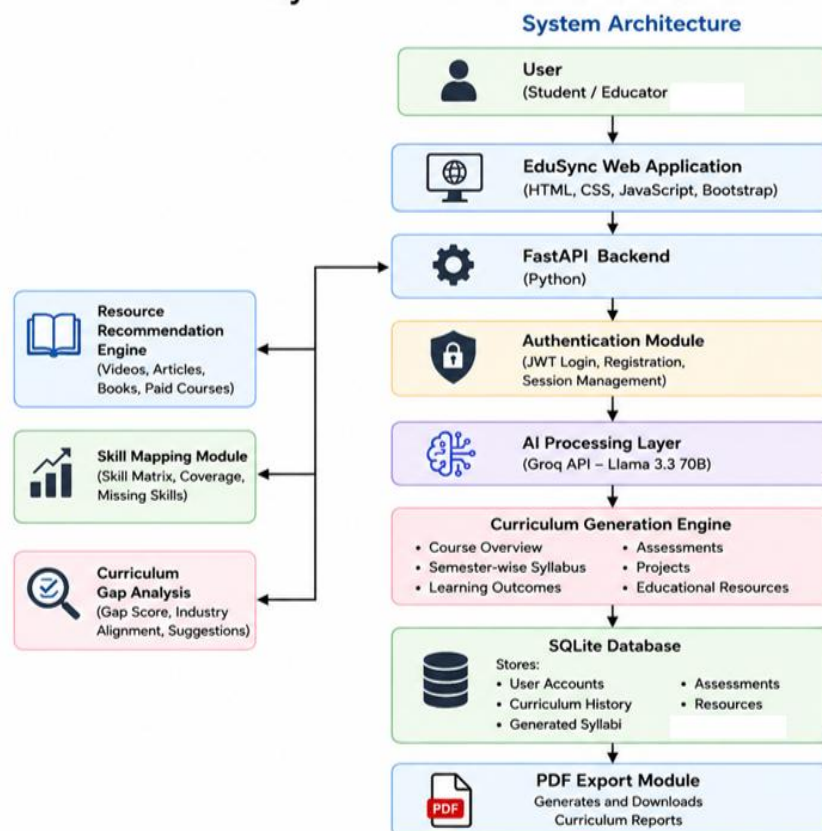
An educator uploads or reviews an existing curriculum to evaluate its effectiveness. EduSync performs AI-based analysis to identify missing topics, outdated concepts, and skill gaps based on modern educational and industry requirements. The system suggests improvements, additional skills, and updated resources to make the curriculum more comprehensive and industry-aligned.

Architecture Overview

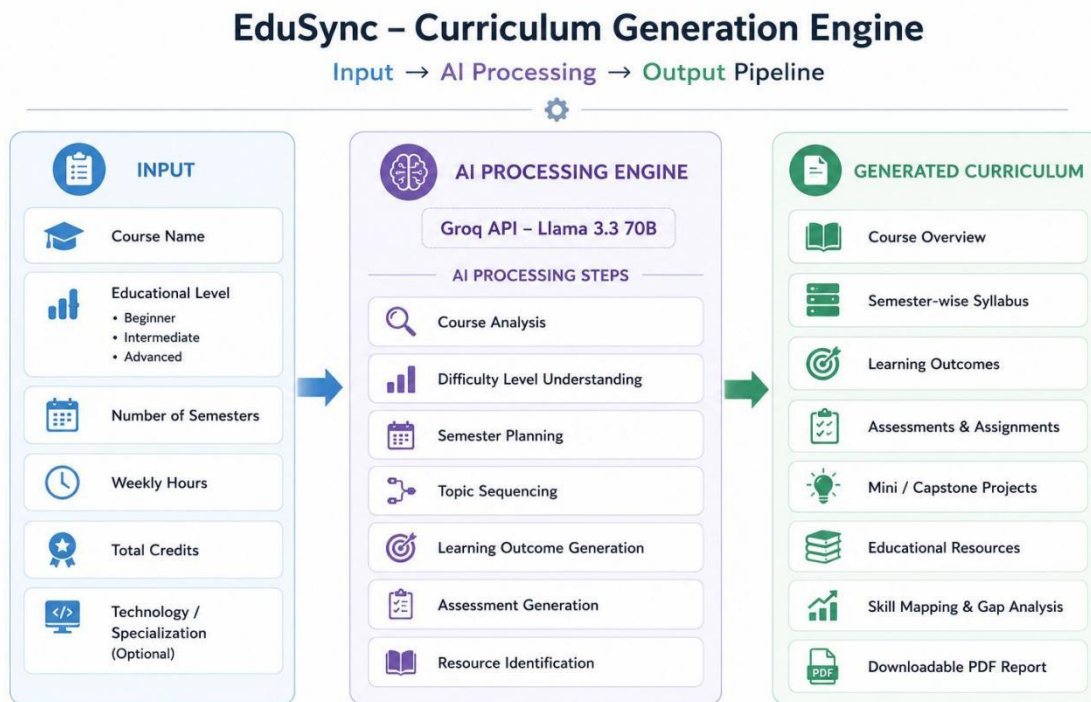
EduSync is designed as a multi-layered web application following a client-server architecture. The system comprises five primary layers: the User Interface Layer, the FastAPI Backend Layer, the AI Processing Layer (Groq API), the Curriculum Generation Engine, and the Data Persistence Layer (SQLite). Each layer has well-defined responsibilities and communicates through secure, structured interfaces.

The architecture ensures separation of concerns, with each layer independently handling its designated responsibilities. The frontend communicates with the backend exclusively through RESTful API endpoints. The backend validates user requests, manages authentication, and orchestrates AI processing. The AI layer generates educational content which is then structured by the Curriculum Generation Engine and stored in the database.

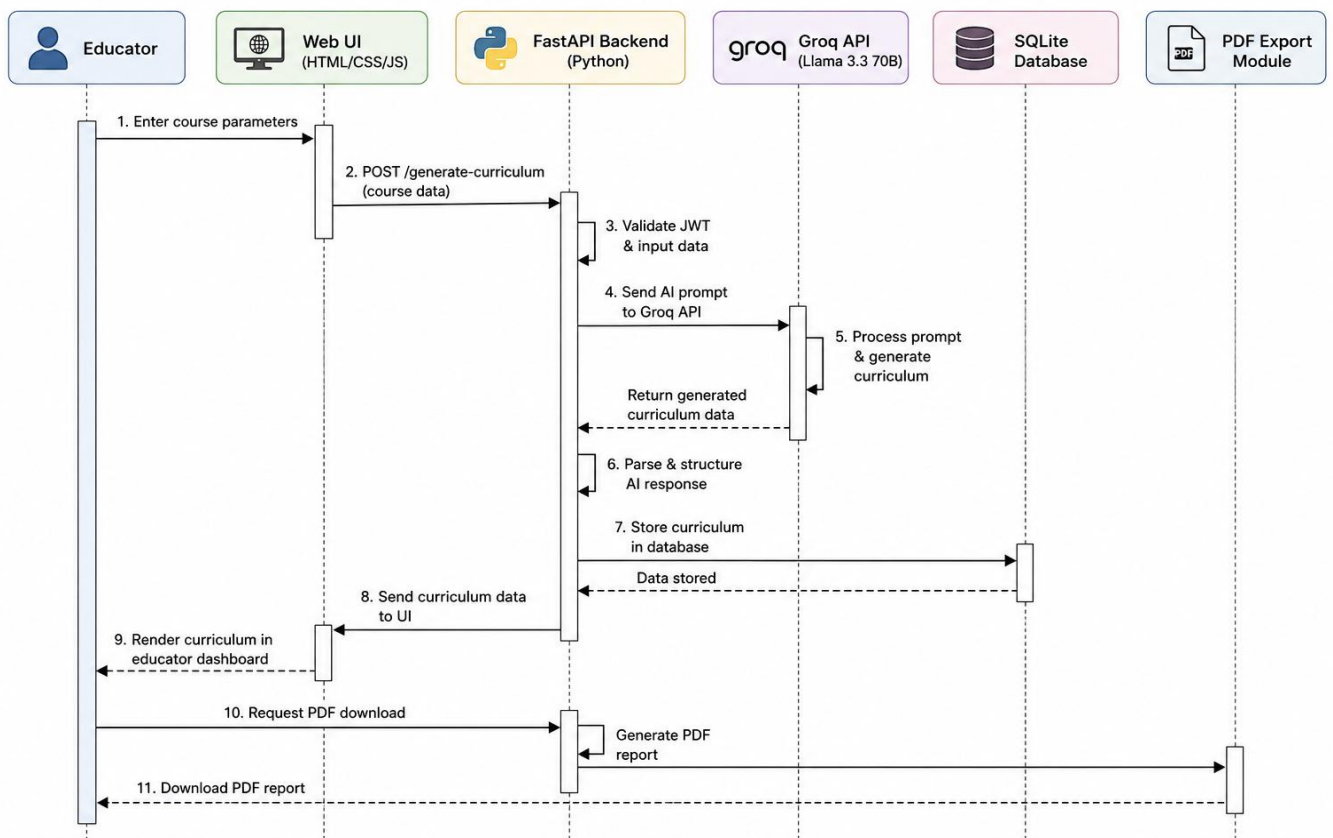
EduSync – Generative AI Powered Curriculum Design System



Educator Workflow Diagram



Sequence Diagram – Curriculum Generation Process



Technology Stack

Frontend Technologies

- **HTML5** – Structure and layout of web pages.
- **CSS3** – Styling, responsiveness, and user interface design.
- **JavaScript** – Client-side interactivity and dynamic content handling.
- **Bootstrap** – Responsive design and reusable UI components.

Backend Technologies

- **Python** – Core programming language for server-side development.
- **FastAPI** – High-performance web framework used to develop RESTful APIs, handle requests, authentication, and AI communication.

Database

- **SQLite** – Lightweight relational database used to store:
 - User information
 - Educator and student data
 - Generated curriculum
 - Weekly learning plans
 - Assessments
 - Educational resources
 - Curriculum history

Artificial Intelligence

- **Groq API** – AI inference platform used to access large language models.
- **Llama 3.3 70B** – Large Language Model used for:
 - Curriculum generation
 - Semester-wise syllabus creation
 - Learning outcome generation
 - Assessment generation
 - Project suggestions
 - Educational resource recommendations
 - Student weekly learning plans

Authentication & Security

- **JWT (JSON Web Token)** – Used for secure authentication, session management, and protected API access.

API Communication

- **REST APIs** – Enable communication between the frontend, FastAPI backend, AI processing layer, and database.

Pre-requisites

Software Requirements

- **Python 3.8+:** Required for developing and running the FastAPI backend and AI integration modules.
- **Groq API Access:** API key required to connect with the Llama 3.3 70B language model for AI-powered curriculum generation.
- **SQLite Database:** Used for storing user details, generated curricula, learning plans, and application data.
- **Git:** Used for version control and project management.
- **Code Editor:** Visual Studio Code, PyCharm, or any preferred IDE.
- **Web Browser:** Google Chrome, Firefox, or any modern browser for accessing the application.

Knowledge Prerequisites

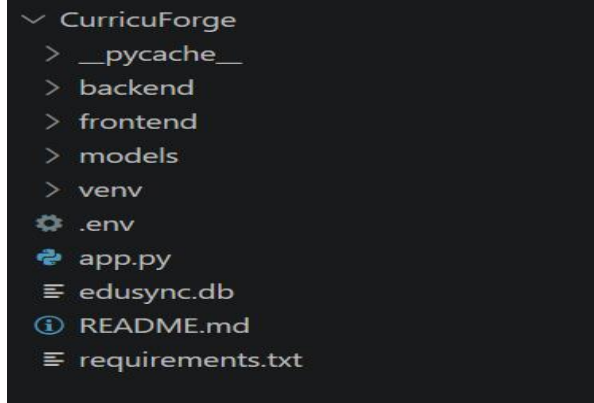
- **Python Basics:** Understanding of functions, object-oriented programming, exception handling, and API development.
- **FastAPI Framework:** Knowledge of routing, request handling, REST APIs, and JSON responses.
- **HTML/CSS:** Understanding of responsive web design, Flexbox, Grid layouts, and styling concepts.
- **JavaScript:** Knowledge of DOM manipulation, asynchronous programming, Fetch API, and frontend interactions.
- **Database Concepts:** Basic understanding of SQLite, database design, and CRUD operations.
- **Generative AI Concepts:** Understanding of Large Language Models (LLMs), prompt engineering, and AI API integration.
- **Educational Domain:** Basic understanding of curriculum design, learning outcomes, and Bloom's Taxonomy.

Hardware Requirements

- **Processor:** Intel Core i3/i5 or AMD Ryzen 3/5 or better.
- **RAM:** Minimum 4GB (8GB recommended for smooth development and execution).
- **Storage:** Minimum 2GB free space for project files and dependencies.
- **Internet Connection:** Required for Groq API communication and AI content generation.

Project Workflow

Project Structure



Milestone 1: Environment Setup & Project Configuration

This milestone establishes the development environment by installing Python dependencies, configuring the FastAPI backend, setting up the SQLite database, and configuring Groq API connectivity for AI-powered curriculum generation.

Activity 1.1: Install Dependencies & Setup Environment

Step 1: Create requirements.txt



Step 2: Create Virtual Environment

```
python -m venv venv
```

Activate virtual environment:

Windows

```
venv\Scripts\activate
```

Mac/Linux

```
source venv/bin/activate
```


Step 3: Install Dependencies

pip install -r requirements.txt

Activity 1.2: Configure Groq API & Environment Variables

Create .env file:

```
CurricuForge > .env
1 GROQ_API_KEY=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
2 SECRET_KEY=abc123xyz
```

Milestone 2: Core Backend Development

This milestone develops the FastAPI backend and integrates Groq Llama 3.3 70B for AI-based curriculum generation. The backend handles user authentication, curriculum generation, student learning plans, resource recommendations, assessments, database management, and PDF generation.

```
from fastapi import FastAPI, Depends, HTTPException
from fastapi.middleware.cors import CORSMiddleware
from fastapi.staticfiles import StaticFiles
from fastapi.responses import FileResponse, StreamingResponse
from pydantic import BaseModel
from pathlib import Path
from groq import RateLimitError, APIError
import io, json

from backend.auth import register_user, login_user, get_current_user
from backend.generator import generate_curriculum
from backend.resources import get_resources
from backend.pdf_generator import build_curriculum_pdf
from backend.educator_generator import generate_educator_curriculum
from backend.educator_pdf import build_educator_pdf
from backend.database import save_course, get_courses, get_course, delete_course

app = FastAPI(title="EduSync")

app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_methods=["*"],
    allow_headers=["*"],
)

FRONTEND = Path(__file__).parent.parent / "frontend"
app.mount("/static", StaticFiles(directory=FRONTEND), name="static")

# --- Auth ---

class RegisterRequest(BaseModel):
    username: str
    email: str
    password: str
    confirm_password: str
    role: str = "student"

class LoginRequest(BaseModel):
    username: str
    password: str

@app.post("/auth/register")
def register(body: RegisterRequest):
    if body.password != body.confirm_password:
        raise HTTPException(status_code=400, detail="Passwords do not match")
    if len(body.password) < 6:
        raise HTTPException(status_code=400, detail="Password must be at least 6 characters")
    return register_user(body.username.strip(), body.email.strip(), body.password, body.role)
```



```

@app.post("/auth/login")
def login(body: LoginRequest):
    return login_user(body.username.strip(), body.password)

# --- Curriculum ---
class CurriculumRequest(BaseModel):
    topic: str
    level: str
    duration: int
    goal: str

@app.post("/generate")
def generate(req: CurriculumRequest, user: dict = Depends(get_current_user)):
    try:
        return generate_curriculum(req.topic, req.level, req.duration, req.goal)
    except (RateLimitError, APIError) as e:
        raise HTTPException(status_code=429 if isinstance(e, RateLimitError) else 502, detail=str(e))

@app.get("/resources")
def resources(topic: str, level: str, user: dict = Depends(get_current_user)):
    try:
        return get_resources(topic, level)
    except (RateLimitError, APIError) as e:
        raise HTTPException(status_code=429 if isinstance(e, RateLimitError) else 502, detail=str(e))

class PDFRequest(BaseModel):
    curriculum: dict
    topic: str
    level: str
    duration: int
    goal: str

@app.post("/download-pdf")
def download_pdf(req: PDFRequest, user: dict = Depends(get_current_user)):
    pdf_bytes = build_curriculum_pdf(req.curriculum, {
        "topic": req.topic, "level": req.level,
        "duration": req.duration, "goal": req.goal
    })
    filename = f"EduSync_{req.topic.replace(' ', '_')}_Curriculum.pdf"
    return StreamingResponse(io.BytesIO(pdf_bytes), media_type="application/pdf",
                             headers={"Content-Disposition": f'attachment; filename="{filename}"'})

# --- Educator ---
class EducatorRequest(BaseModel):
    subject: str
    level: str
    semesters: int
    weekly_hours: int
    industry_focus: str
  
```

```

@app.post("/educator/generate")
def educator_generate(req: EducatorRequest, user: dict = Depends(get_current_user)):
    try:
        return generate_educator_curriculum(
            req.subject, req.level, req.semesters,
            req.weekly_hours, req.industry_focus
        )
    except (RateLimitError, APIError) as e:
        raise HTTPException(status_code=429 if isinstance(e, RateLimitError) else 502, detail=str(e))

class EducatorPDFRequest(BaseModel):
    curriculum: dict
    subject: str
    level: str
    semesters: int
    weekly_hours: int
    industry_focus: str

@app.post("/educator/download-pdf")
def educator_download_pdf(req: EducatorPDFRequest, user: dict = Depends(get_current_user)):
    pdf_bytes = build_educator_pdf(req.curriculum, {
        "subject": req.subject, "level": req.level,
        "semesters": req.semesters, "weekly_hours": req.weekly_hours,
        "industry_focus": req.industry_focus
    })
    filename = f"EduSync_{req.subject.replace(' ', '_')}_Educator_Curriculum.pdf"
    return StreamingResponse(io.BytesIO(pdf_bytes), media_type="application/pdf",
                             headers={"Content-Disposition": f'attachment; filename="{filename}"'})

# --- Saved Courses (SQLite) ---

class SaveCourseRequest(BaseModel):
    course_id: str
    title: str
    subject: str
    level: str
    role_type: str
    meta: dict
    full_data: dict

@app.post("/courses/save")
def course_save(req: SaveCourseRequest, user: dict = Depends(get_current_user)):
    save_course(
        user_id=user["id"], course_id=req.course_id,
        title=req.title, subject=req.subject, level=req.level,
        role_type=req.role_type, meta=req.meta, full_data=req.full_data
    )
    return {"message": "Course saved"}
  
```

```
@app.get("/courses")
def courses_list(user: dict = Depends(get_current_user)):
    rows = get_courses(user["id"])
    for r in rows:
        r["meta"] = json.loads(r.get("meta_json") or "{}")
        r["full_data"] = json.loads(r.get("full_json") or "{}")
        r.pop("meta_json", None)
        r.pop("full_json", None)
    return rows

@app.get("/courses/{course_id}")
def course_detail(course_id: str, user: dict = Depends(get_current_user)):
    row = get_course(user["id"], course_id)
    if not row:
        raise HTTPException(status_code=404, detail="Course not found")
    row["meta"] = json.loads(row.get("meta_json") or "{}")
    row["full_data"] = json.loads(row.get("full_json") or "{}")
    row.pop("meta_json", None)
    row.pop("full_json", None)
    return row

@app.delete("/courses/{course_id}")
def course_delete(course_id: str, user: dict = Depends(get_current_user)):
    delete_course(user["id"], course_id)
    return {"message": "Course deleted"}

# --- Week Quiz ---

class QuizRequest(BaseModel):
    week: int
    title: str
    topics: list[str]
    level: str

@app.post("/quiz/generate")
def generate_week_quiz(req: QuizRequest, user: dict = Depends(get_current_user)):
    from backend.generator import client, MODEL
    prompt = (
        f"Generate exactly 25 multiple-choice quiz questions to assess a {req.level} student's understanding "
        f"of Week {req.week}: '{req.title}'.\n"
        f"Topics covered: {', '.join(req.topics)}.\n"
        f"Cover all topics thoroughly with varied difficulty (easy, medium, hard).\n"
        f"Return a JSON array of exactly 25 objects only - no extra text:\n"
        f"[{{\"question\": \"...\", \"options\": [\"A\", \"B\", \"C\", \"D\"], \"answer\": 0, \"explanation\": \"...\"}}]"
    )
    try:
        resp = client.chat.completions.create(
            model=MODEL,
            messages=[{"role": "user", "content": prompt}],
            temperature=0.7,
            max_tokens=4000,
        )
        text = resp.choices[0].message.content.strip()
```

```
# --- Chatbot ---

class ChatRequest(BaseModel):
    message: str
    context: dict = {} # {topic, level, course_title, current_week, weekly_plan}
    history: list = [] # [{role, content}, ...] last N turns

@app.post("/chatbot")
def chatbot(req: ChatRequest, user: dict = Depends(get_current_user)):
    from backend.generator import client, MODEL

    ctx = req.context
    system = (
        "You are EduBot, an AI academic assistant embedded in CurricuForge - an AI-powered curriculum platform.\n"
        "Your ONLY purpose is to help students and educators with EDUCATIONAL topics.\n"
        "You may answer questions about:\n"
        "  • The course curriculum, weekly topics, learning outcomes, and study plans shown on this platform\n"
        "  • Subject-matter concepts related to the course topic\n"
        "  • Study strategies, exam preparation, and learning best practices\n"
        "  • How to navigate and use CurricuForge features (generate curriculum, quizzes, PDF download, dashboard)\n"
        "You must REFUSE to answer anything unrelated to education - including general chat, personal advice, "
        "coding unrelated to coursework, politics, entertainment, or any off-topic requests.\n"
        "When refusing, say: 'I can only help with educational topics and this platform. Please ask a course or study-related question.'\n"
    )

    if ctx.get("course_title"):
        system += f"\nCurrent course: {ctx['course_title']}"
    if ctx.get("topic"):
        system += f" | Topic: {ctx['topic']}"
    if ctx.get("level"):
        system += f" | Level: {ctx['level']}"
    if ctx.get("current_week"):
        system += f" | Student is on Week {ctx['current_week']}"
    if ctx.get("weekly_plan"):
        plan_summary = "; ".join(
            f"Wk{w.get('week', '?')}: {w.get('title', '')}"
            for w in ctx["weekly_plan"][:8]
        )
        system += f"\nCourse weekly plan (first 8 weeks): {plan_summary}"

    messages = [{"role": "system", "content": system}]
    for h in req.history[-6:]: # last 6 turns for context
        if h.get("role") in ("user", "assistant"):
            messages.append({"role": h["role"], "content": h["content"]})
    messages.append({"role": "user", "content": req.message})

    try:
        resp = client.chat.completions.create(
            model=MODEL,
            messages=messages,
            temperature=0.5,
            max_tokens=600,
        )
```

```

    return {"reply": resp.choices[0].message.content.strip()}
  except (RateLimitError, APIError) as e:
    raise HTTPException(status_code=429 if isinstance(e, RateLimitError) else 502, detail=str(e))

# — Frontend pages —

@app.get("/")
def root():
    return FileResponse(FRONTEND / "login.html")

@app.get("/{page}")
def pages(page: str):
    f = FRONTEND / f"{page}.html"
    if f.exists():
        return FileResponse(f)
    return FileResponse(FRONTEND / "login.html")

```

Milestone 3: Frontend Development

This milestone develops a responsive web interface using HTML,CSS,JavaScript and Bootstrap.

Activity 3.1: User Interface Development

Authentication Page

```

// — Login / Register toggle —
let isLogin = true;

function setMode(login) {
    isLogin = login;
    document.getElementById("auth-title").textContent = login ? "Welcome Back" : "Create Account";
    document.getElementById("auth-sub").textContent = login ? "Sign in to continue" : "Join EduSync today";
    document.getElementById("btn-text").textContent = login ? "Sign In" : "Register";
    document.getElementById("toggle-link").textContent = login ? "Register" : "Sign In";
    document.getElementById("toggle-link").previousSibling.textContent = login
    ? "Don't have an account?" : "Already have an account?";

    document.getElementById("field-email").classList.toggle("hidden", login);
    document.getElementById("field-confirm").classList.toggle("hidden", login);

    // email & confirm_password required only on register
    document.querySelector("input[name='email']").required = !login;
    document.querySelector("input[name='confirm_password']").required = !login;

    document.getElementById("auth-alert").className = "alert";
    document.getElementById("auth-alert").textContent = "";
}

document.getElementById("toggle-link").addEventListener("click", () => setMode(!isLogin));

// — Form submit —
document.getElementById("auth-form").addEventListener("submit", async (e) => {
    e.preventDefault();
    const alertEl = document.getElementById("auth-alert");
    alertEl.className = "alert"; alertEl.textContent = "";
});

```

```

async function renderDashboard() {
    let courses = [];
    try {
        courses = await dbGetCourses();
    } catch (err) {
        document.getElementById("courses-list").innerHTML = `<p style="color:var(--danger)">Failed to load courses: ${err.message}</p>`;
        return;
    }

    courses.forEach(c => {
        c.topic = (c.meta && c.meta.topic) || c.subject || "";
        c.totalWeeks = (c.meta && c.meta.totalWeeks) || 1;
        c.weeksDone = (c.meta && c.meta.weeksDone) || 0;
    });

    const total = courses.length;
    const done = courses.filter(c => c.weeksDone >= c.totalWeeks).length;
    const inProg = courses.filter(c => c.weeksDone > 0 && c.weeksDone < c.totalWeeks).length;
    const avgPct = total ? Math.round(courses.reduce((s,c) => s + (c.weeksDone/c.totalWeeks)*100, 0) / total) : 0;

    document.getElementById("stats-row").innerHTML = `
    <div class="stat-card"><div class="stat-val">${total}</div><div class="stat-label">Courses Enrolled</div></div>
    <div class="stat-card"><div class="stat-val">${done}</div><div class="stat-label">Completed</div></div>
    <div class="stat-card"><div class="stat-val">${inProg}</div><div class="stat-label">In Progress</div></div>
    <div class="stat-card"><div class="stat-val">${avgPct}</div><div class="stat-label">Avg. Progress</div></div>`;

    document.getElementById("overall-pct").textContent = avgPct + "%";
    document.getElementById("overall-bar").style.width = avgPct + "%";

    const list = document.getElementById("courses-list");
    if (!total) {

```



```

let _data = null, _req = null,

// --- Tab switch ---
function switchTab(name, btn) {
  document.querySelectorAll("#main-tabs .tab-btn").forEach(b => b.classList.remove("active"));
  document.querySelectorAll(".tab-panel").forEach(p => p.classList.remove("active"));
  btn.classList.add("active");
  document.getElementById("tab-" + name).classList.add("active");
}

// --- Form submit ---
document.getElementById("gen-form").addEventListener("submit", async (e) => {
  e.preventDefault();
  const alertEl = document.getElementById("gen-alert");
  alertEl.textContent = ""; alertEl.className = "alert";
  const f = e.target;
  _req = {
    subject:      f.subject.value.trim(),
    level:        f.level.value,
    semesters:    parseInt(f.semesters.value),
    weekly_hours: parseInt(f.weekly_hours.value),
    industry_focus: f.industry_focus.value
  };
  document.getElementById("loader").style.display = "block";
  document.getElementById("results").style.display = "none";
  document.getElementById("gen-btn").disabled = true;

  // Rotate loader messages
  const msgs = [
    "Generating course overview...",
    "Building semester structure...",
    "Creating weekly plans...",
    "Generating assessments..."
  ];

```

```

function switchTab(name, btn) {
  document.querySelectorAll(".tab-btn").forEach(b => b.classList.remove("active"));
  document.querySelectorAll(".tab-panel").forEach(p => p.classList.remove("active"));
  btn.classList.add("active");
  document.getElementById("tab-" + name).classList.add("active");
}

document.getElementById("gen-form").addEventListener("submit", async (e) => {
  e.preventDefault();
  const alertEl = document.getElementById("gen-alert");
  alertEl.textContent = ""; alertEl.className = "alert";
  const form = e.target;
  _currentReq = { topic: form.topic.value.trim(), level: form.level.value, duration: parseInt(form.duration.value), goal: form.goal.value };
  document.getElementById("loader").style.display = "block";
  document.getElementById("results").style.display = "none";
  document.getElementById("gen-btn").disabled = true;
  try {
    _currentData = await apiFetch("/generate", { method: "POST", body: JSON.stringify(_currentReq) });
    renderResults(_currentData, _currentReq);
  } catch (err) {
    alertEl.className = "alert alert-error";
    alertEl.textContent = err.message;
  } finally {
    document.getElementById("loader").style.display = "none";
    document.getElementById("gen-btn").disabled = false;
  }
});

function renderResults(d, req) {

```

Milestone 4: Application Deployment & Execution

This milestone focuses on running the complete EduSync system locally and verifying all modules.

Start FastAPI Backend

uvicorn backend.main:app --reload

```

PS C:\p\CurricuForge> python -m uvicorn app:app --reload --host 127.0.0.1 --port 8000
INFO: Will watch for changes in these directories: ['C:\p\CurricuForge']
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO: Started reloader process [32752] using WatchFiles
INFO: Started server process [33320]
INFO: Waiting for application startup.

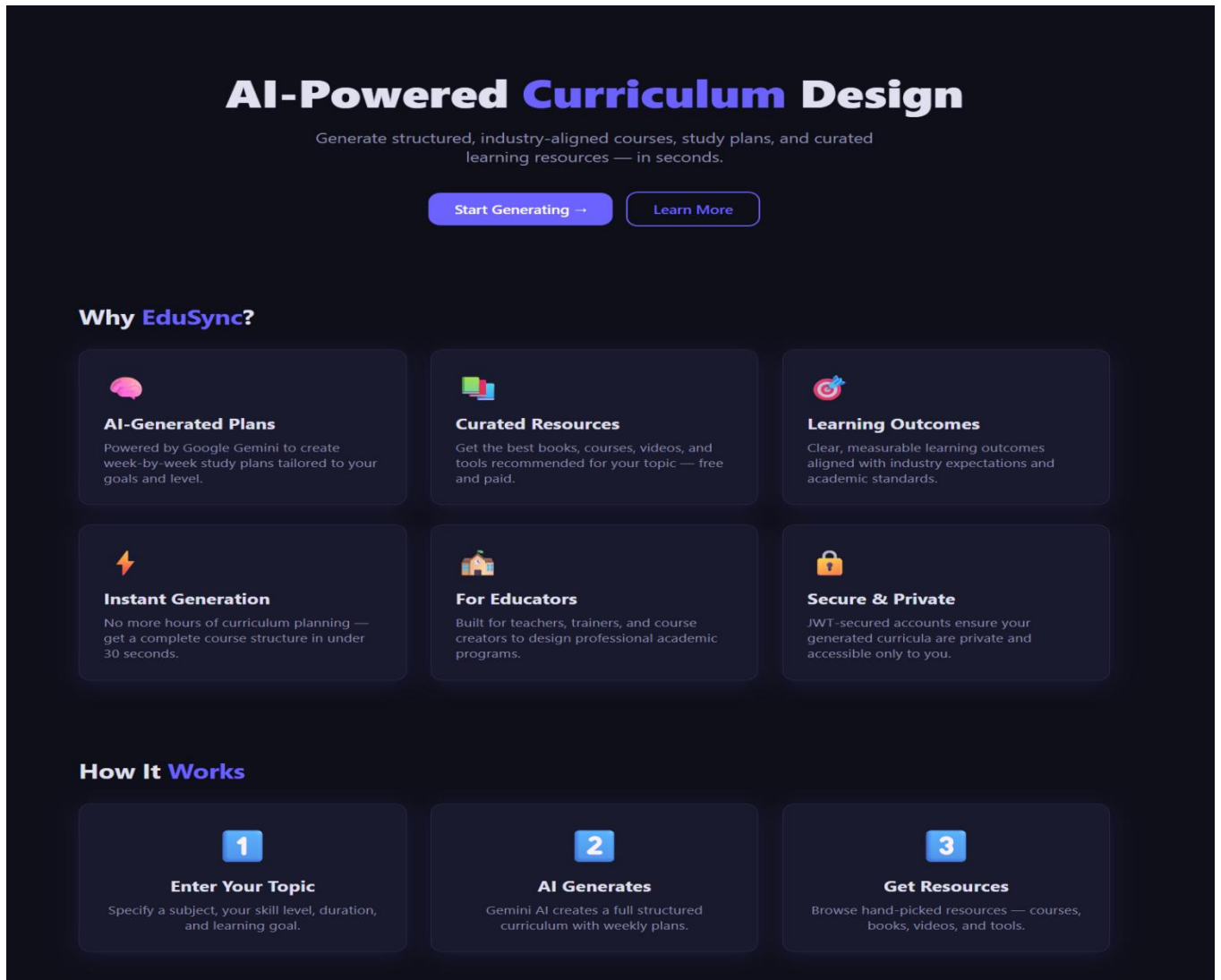
```

Access the Web Application

<http://127.0.0.1:8000>

Project Output

Home Page



The screenshot shows the home page of the 'AI-Powered Curriculum Design' application. The page has a dark blue background with white and light blue text. At the top, the title 'AI-Powered Curriculum Design' is prominently displayed in a large, bold font. Below the title, a subtitle reads: 'Generate structured, industry-aligned courses, study plans, and curated learning resources — in seconds.' Two buttons are positioned below the subtitle: 'Start Generating →' in a light blue box and 'Learn More' in a white box. The main content area is divided into two sections. The first section, 'Why EduSync?', features six cards arranged in a 2x3 grid. Each card has an icon, a title, and a brief description. The cards are: 'AI-Generated Plans' (brain icon), 'Curated Resources' (book icon), 'Learning Outcomes' (target icon), 'Instant Generation' (lightning bolt icon), 'For Educators' (house icon), and 'Secure & Private' (lock icon). The second section, 'How It Works', features three cards arranged horizontally, numbered 1, 2, and 3. Card 1 is 'Enter Your Topic', Card 2 is 'AI Generates', and Card 3 is 'Get Resources'. Each card provides a brief description of the step.

AI-Powered Curriculum Design

Generate structured, industry-aligned courses, study plans, and curated learning resources — in seconds.

[Start Generating →](#) [Learn More](#)

Why EduSync?

- AI-Generated Plans**
Powered by Google Gemini to create week-by-week study plans tailored to your goals and level.
- Curated Resources**
Get the best books, courses, videos, and tools recommended for your topic — free and paid.
- Learning Outcomes**
Clear, measurable learning outcomes aligned with industry expectations and academic standards.
- Instant Generation**
No more hours of curriculum planning — get a complete course structure in under 30 seconds.
- For Educators**
Built for teachers, trainers, and course creators to design professional academic programs.
- Secure & Private**
JWT-secured accounts ensure your generated curricula are private and accessible only to you.

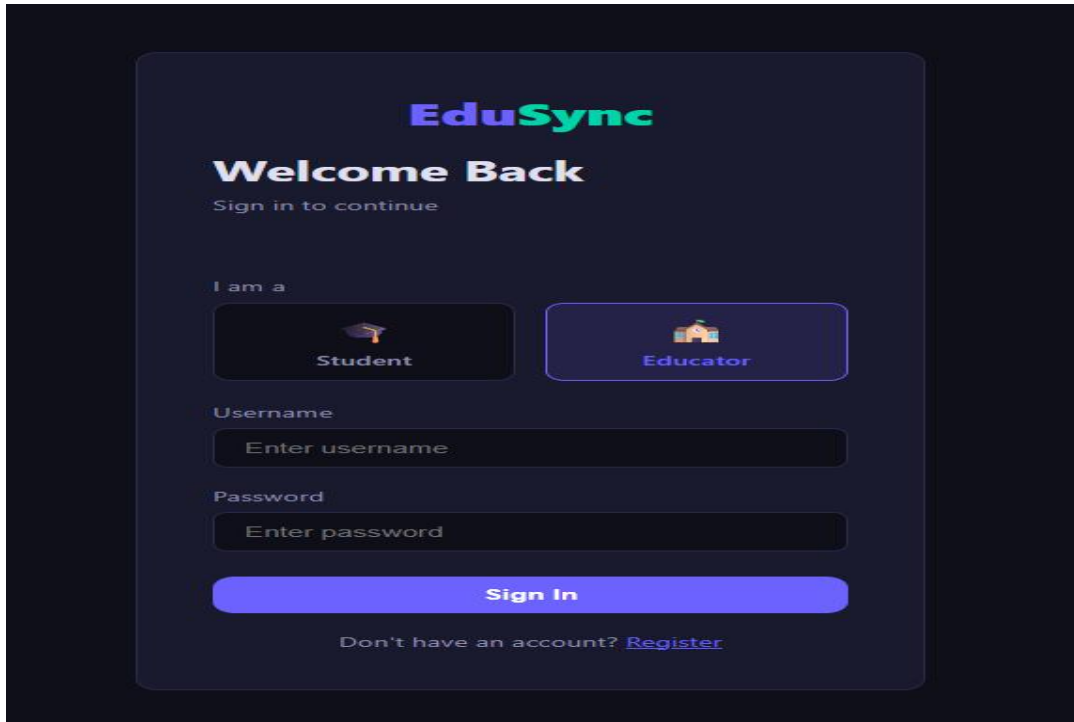
How It Works

- 1 Enter Your Topic**
Specify a subject, your skill level, duration, and learning goal.
- 2 AI Generates**
Gemini AI creates a full structured curriculum with weekly plans.
- 3 Get Resources**
Browse hand-picked resources — courses, books, videos, and tools.

Login Page

The Educator Login page provides a secure login interface with email and password fields. JWT authentication is initiated upon successful credential validation.

The Student Login page mirrors the educator login interface with appropriate role-based routing upon authentication.



EduSync

Welcome Back

Sign in to continue

I am a


Student


Educator

Username

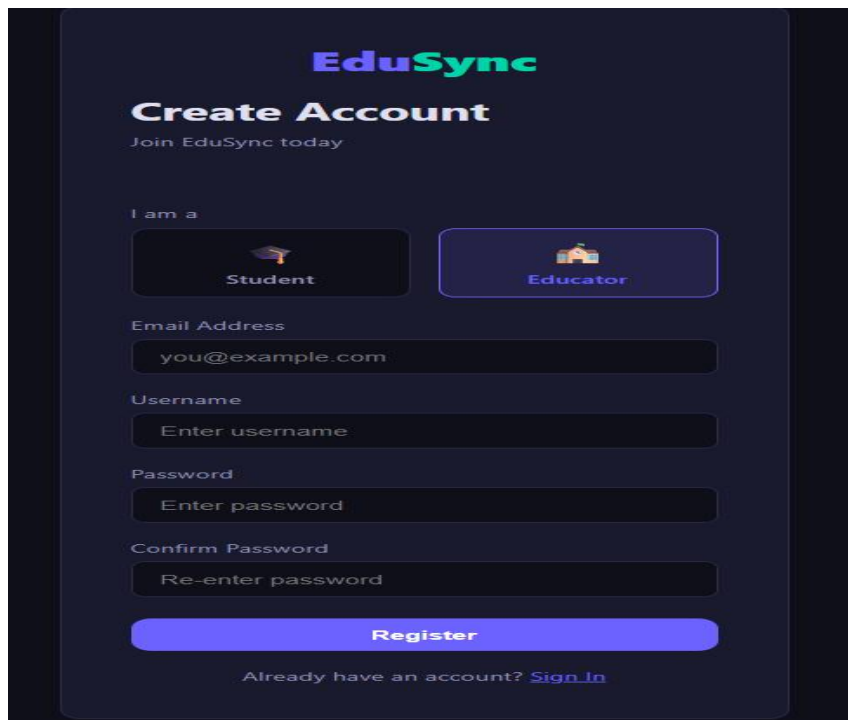
Password

Sign In

Don't have an account? [Register](#)

Registration Page

The Registration Page allows new users to register with name, email, password, and role selection (Educator or Student).



EduSync

Create Account

Join EduSync today

I am a


Student


Educator

Email Address

Username

Password

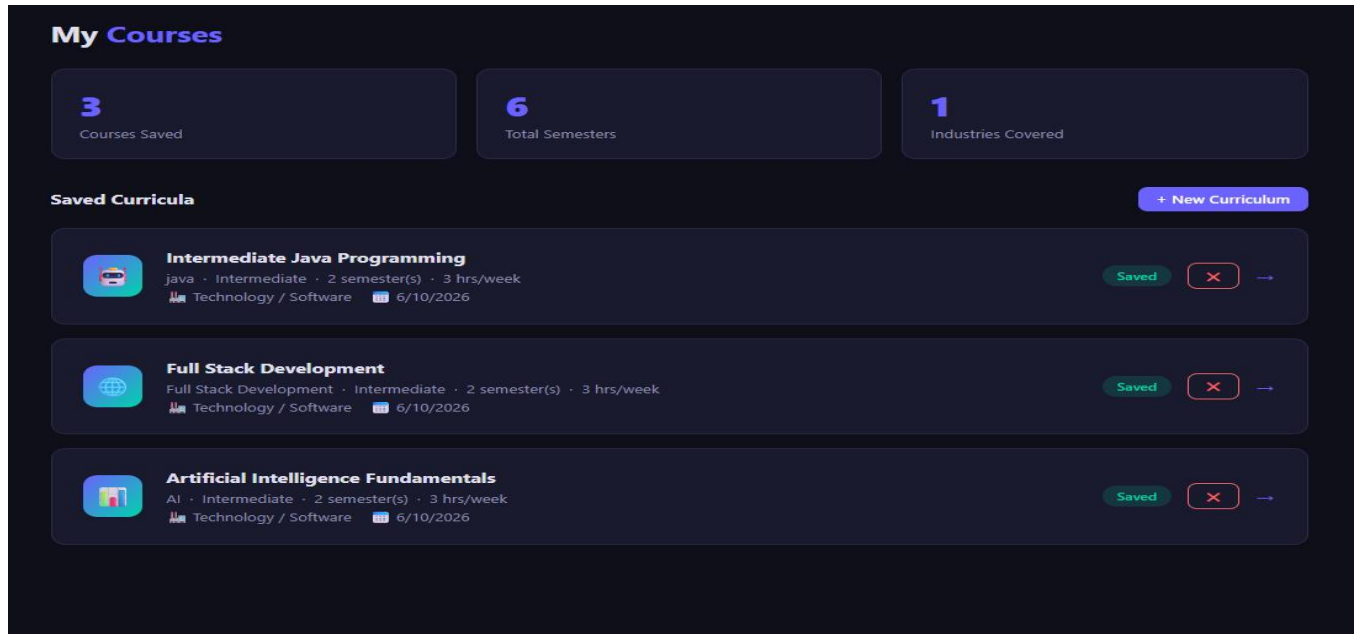
Confirm Password

Register

Already have an account? [Sign In](#)

Educator Dashboard

The Educator Dashboard displays the user's generated curricula history, quick access to the curriculum generation module, analytics summary cards, and navigation to all educator features.



My Courses

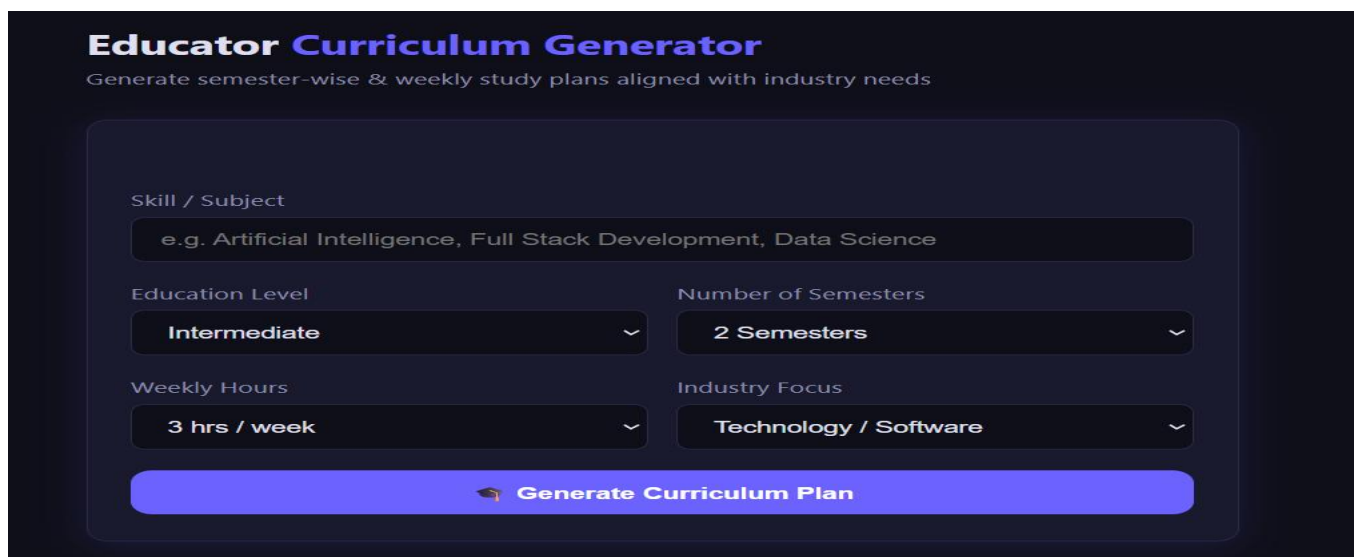
- 3** Courses Saved
- 6** Total Semesters
- 1** Industries Covered

Saved Curricula [+ New Curriculum](#)

- 
Intermediate Java Programming
 java · Intermediate · 2 semester(s) · 3 hrs/week
 Technology / Software 6/10/2026 Saved ✕ →
- 
Full Stack Development
 Full Stack Development · Intermediate · 2 semester(s) · 3 hrs/week
 Technology / Software 6/10/2026 Saved ✕ →
- 
Artificial Intelligence Fundamentals
 AI · Intermediate · 2 semester(s) · 3 hrs/week
 Technology / Software 6/10/2026 Saved ✕ →

Curriculum Generation Page

The Curriculum Generation Page presents the course input form where educators enter Course Name, Educational Level, Number of Semesters, Weekly Hours, Total Credits, and optional Technology Specialization. A Generate Curriculum button triggers the AI processing.



Educator Curriculum Generator
 Generate semester-wise & weekly study plans aligned with industry needs

Skill / Subject

Education Level Number of Semesters

Intermediate **2 Semesters**

Weekly Hours Industry Focus

3 hrs / week **Technology / Software**

 **Generate Curriculum Plan**

Generated Curriculum Output

The Generated Curriculum Output displays the complete AI-generated curriculum in a structured, semester-wise accordion layout. Each semester panel expands to show course names, modules, topics.

Semester Plan
Outcome Mapping
Industry Projects
Resources

Semester 1: Front-end Development
This semester focuses on front-end development, covering HTML, CSS, and JavaScript.
Credits: 4
Total: 48 hrs

OUTCOMES COVERED
LO1 LO2

16-Week Plan

Week	Topic	Subtopics	Method	Hours	Assessment
W1	Introduction to Front-end Development	Overview of Front-end Development · Importance of HTML, CSS, and JavaScript · Setting up the Development Environment	Lecture	3h	—
W2	HTML Fundamentals	Basic HTML Structure · HTML Elements and Attributes · Semantic HTML	Lab	3h	Lab
W3	HTML Advanced Topics	HTML Forms · HTML Tables · HTML Multimedia	Lecture	3h	Quiz
W4	CSS Fundamentals	Introduction to CSS · CSS Selectors and Properties · CSS Box Model	Lecture	3h	Assignment
W5	CSS Advanced Topics	CSS Positioning · CSS Flexbox · CSS Grid	Lab	3h	Lab
W6	CSS Preprocessors and Frameworks	Introduction to Sass and Less · Using Bootstrap and Materialize	Lecture	3h	—
W7	JavaScript Fundamentals	Introduction to JavaScript · JavaScript Variables and Data Types · JavaScript Control Structures	Lecture	3h	Quiz
W8	JavaScript Advanced Topics	JavaScript Functions · JavaScript Objects and Arrays · JavaScript DOM Manipulation	Lab	3h	Lab
W9	JavaScript Libraries and Frameworks	Introduction to jQuery and React · Using Angular and Vue.js	Lecture	3h	Lab
W10	Front-end Development Best Practices	Code Organization and Readability · Performance Optimization · Accessibility and Security	Lecture	3h	Assignment
W11	Responsive Web Design	Introduction to Responsive Web Design · Media Queries and Breakpoints · Mobile-First Design	Lab	3h	Lab
W12	Front-end Build Tools and Workflow	Introduction to Webpack and Gulp · Using npm and yarn	Lecture	3h	Quiz
W13	Front-end Testing and Debugging	Introduction to Jest and Mocha · Using Chrome DevTools	Lab	3h	Lab
W14	Project Planning and Management	Introduction to Agile Methodologies · Using Trello and Asana	Lecture	3h	—
W15	Project Development and Deployment	Introduction to GitHub and Netlify · Deploying a Front-end Project	Lab	3h	Assignment
W16	Final Project Presentations	Final Project Demos · Peer Feedback and Review	Project	3h	Project

Semester Assessments

Assignments

Title	Due	Marks
HTML and CSS Assignment	Wk 4	10m
JavaScript Assignment	Wk 10	10m
Front-end Project Assignment	Wk 15	10m

Quizzes

Title	Week	Marks
HTML and CSS Quiz	Wk 3	5m
JavaScript Quiz	Wk 7	5m
Front-end Development Quiz	Wk 12	5m

Projects

Title	Type	Marks
Group Project	Group	20m
Individual Project	Individual	15m

Lab Exercises

Title	Week	Tools
HTML and CSS Lab	Wk 2	HTML, CSS
JavaScript Lab	Wk 5	JavaScript
Front-end Development Lab	Wk 9	HTML, CSS, JavaScript
Responsive Web Design Lab	Wk 13	HTML, CSS, JavaScript

Mark Distribution

20

Assignments

15

Quizzes

25

Midterm

30

Project

10

Lab

100

Total

Semester 2: Back-end Development and Full Stack Application Development

This semester focuses on back-end development and full stack application development, covering server-side programming, database management, and application deployment.

Credits: 4
Total: 48 hrs

LO	Outcome	Bloom's Level	Semesters	Topics	Assessed By
L01	Understand the basics of front-end development	Remember	S1	HTML, CSS, JavaScript	Quiz Assignment
L02	Understand the basics of back-end development	Understand	S1	Server-side programming, Database management	Quiz Assignment
L03	Apply front-end and back-end development skills to build a full stack application	Apply	S2	Full stack application development, Application deployment	Project Presentation
L04	Analyze and debug full stack applications	Analyze	S2	Debugging techniques, Error handling	Quiz Assignment
L05	Evaluate the performance and security of full stack applications	Evaluate	S2	Performance optimization, Security measures	Quiz Assignment
L06	Create a full stack application from scratch	Create	S2	Full stack application development, Application deployment	Project Presentation

Student Weekly Learning Plan

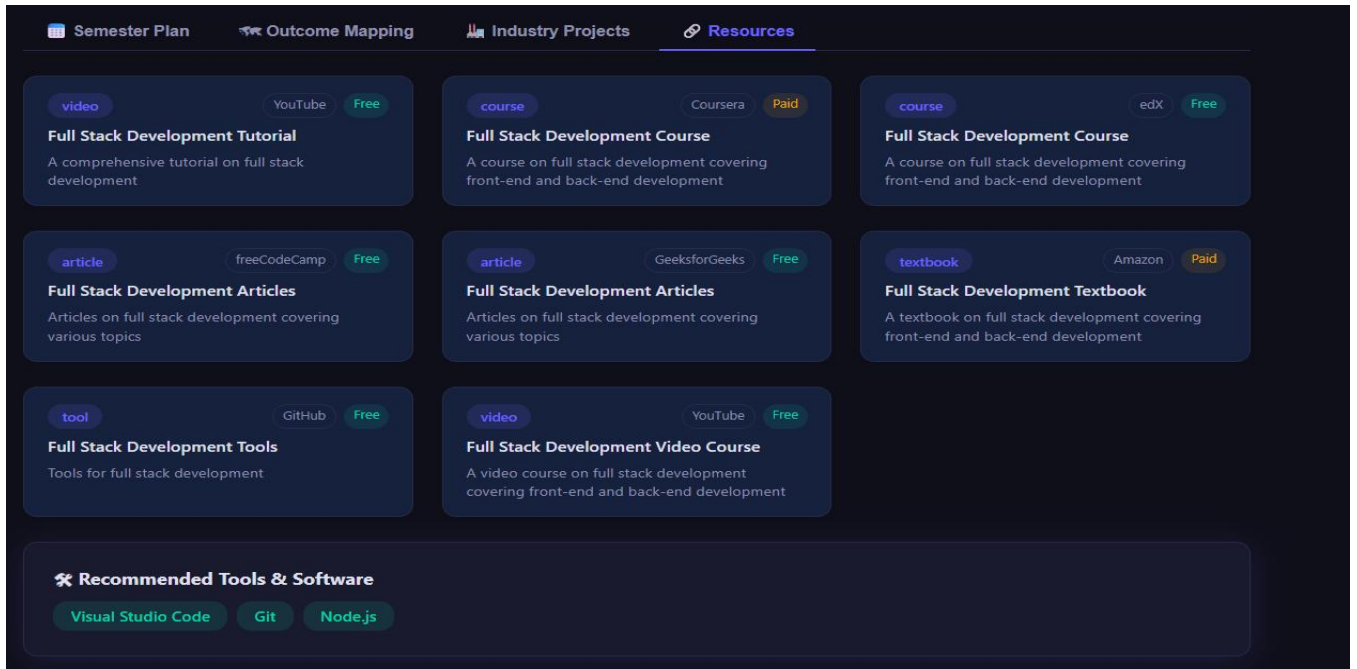
The Student Weekly Plan page displays the AI-generated personalized study schedule with weekly objectives, topics, activities, resources, and mini-assessments presented in a clear week-by-week timeline.

All Topics	Weekly Plan	Resources	Assessment
Introduction to Python			
Introduction to Python Overview of Python and its features	Setting up Python Environment Installing Python and setting up the development environment	Basic Syntax Variables, data types, and basic operators in Python	
Indentation and Comments Using indentation and comments in Python code	Basic Input/Output Reading and writing data in Python	Basic Operators Arithmetic, comparison, and logical operators in Python	
Control Structures			
If-Else Statements Using if-else statements to control the flow of a program	For Loops Using for loops to iterate over sequences	While Loops Using while loops to repeat a block of code	
Break and Continue Statements Using break and continue statements to control the flow of a loop	Pass Statement Using the pass statement as a placeholder	Exception Handling Handling errors and exceptions in Python	
Functions and Modules			
Defining Functions Defining and using functions in Python	Function Arguments Passing arguments to functions in Python	Return Statements Using return statements to return values from functions	
Lambda Functions Using lambda functions to define small, anonymous functions	Modules and Imports Using modules and imports to organize code	Package Management Managing packages and dependencies in Python	
Data Structures			
Lists Using lists to store and manipulate data in Python	Tuples Using tuples to store and manipulate data in Python	Dictionaries Using dictionaries to store and manipulate data in Python	
Sets Using sets to store and manipulate data in Python	Strings Using strings to store and manipulate text data in Python	File Input/Output Reading and writing data to files in Python	
Object-Oriented Programming			

Week 1 Introduction to Python	
TOPICS COVERED - Introduction to Python - Setting up Python Environment - Basic Syntax	ACTIVITIES - Installing Python - Writing first Python program
Week 2 Control Structures	
TOPICS COVERED - If-Else Statements - For Loops - While Loops	ACTIVITIES - Writing programs with if-else statements - Using for loops to iterate over sequences
Week 3 Functions and Modules	
TOPICS COVERED - Defining Functions - Function Arguments - Return Statements	ACTIVITIES - Defining and using functions - Importing modules and using functions
Week 4 Data Structures	
TOPICS COVERED - Lists - Tuples - Dictionaries	ACTIVITIES - Using lists to store and manipulate data - Using dictionaries to store and manipulate data

Resources Page

The Resources Page displays AI-recommended educational resources organized by category (Videos, Articles, Books, Paid Courses) with titles and access links.



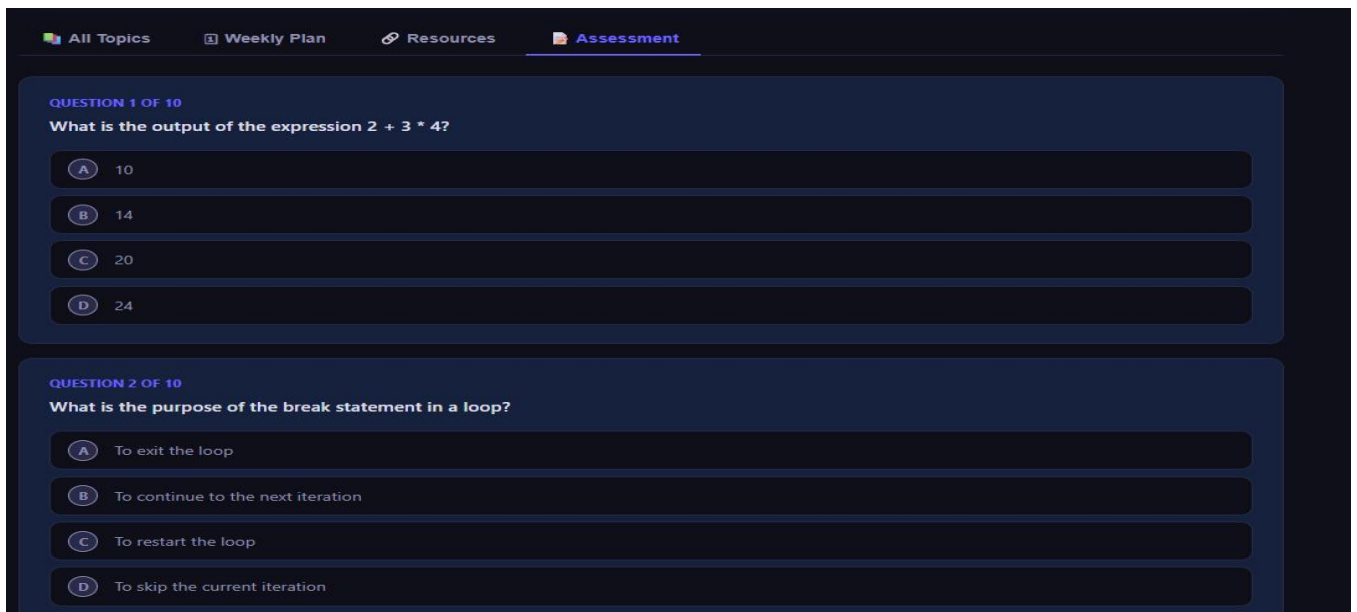
The screenshot shows the 'Resources' tab selected in a navigation bar. Below the navigation bar, there are six resource cards arranged in a 3x2 grid. Each card includes a category label (video, course, article, textbook, tool), a source (YouTube, Coursera, edX, freeCodeCamp, GeeksforGeeks, Amazon, GitHub), and a status (Free, Paid). The resources are:

- Full Stack Development Tutorial** (video, YouTube, Free): A comprehensive tutorial on full stack development.
- Full Stack Development Course** (course, Coursera, Paid): A course on full stack development covering front-end and back-end development.
- Full Stack Development Course** (course, edX, Free): A course on full stack development covering front-end and back-end development.
- Full Stack Development Articles** (article, freeCodeCamp, Free): Articles on full stack development covering various topics.
- Full Stack Development Articles** (article, GeeksforGeeks, Free): Articles on full stack development covering various topics.
- Full Stack Development Textbook** (textbook, Amazon, Paid): A textbook on full stack development covering front-end and back-end development.
- Full Stack Development Tools** (tool, GitHub, Free): Tools for full stack development.
- Full Stack Development Video Course** (video, YouTube, Free): A video course on full stack development covering front-end and back-end development.

At the bottom, there is a section titled 'Recommended Tools & Software' with buttons for Visual Studio Code, Git, and Node.js.

Assessment Page

The Assessment Page presents AI-generated quizzes, assignments, and exam questions for the selected course. Students can attempt assessments and view feedback.



The screenshot shows the 'Assessment' tab selected in a navigation bar. Below the navigation bar, there are two question cards. Each card includes a question number and a question text. The questions are:

- QUESTION 1 OF 10**
What is the output of the expression $2 + 3 * 4$?
Options: (A) 10, (B) 14, (C) 20, (D) 24
- QUESTION 2 OF 10**
What is the purpose of the break statement in a loop?
Options: (A) To exit the loop, (B) To continue to the next iteration, (C) To restart the loop, (D) To skip the current iteration

PDF Export Output

The PDF Export Output shows a sample downloaded curriculum report with professional formatting, semester-wise course tables, learning outcomes, assessment schedules, and resource

EduSync

Educator Curriculum Plan

Intermediate Java Programming

Level: Intermediate	Semesters: 2	Weekly Hours: 3 hrs	Industry: Technology / Software	Code: CS101
---------------------	--------------	---------------------	---------------------------------	-------------

This course covers intermediate-level Java programming concepts, including data structures, file input/output, and object-oriented programming. Students will learn to design, implement, and test Java programs. The course focuses on practical applications and industry-relevant projects.

■ The Java programming language is widely used in the Technology and Software industry for developing large-scale applications, Android apps, and web applications. This course aligns with industry needs by providing students with hands-on experience in Java programming, preparing them for careers in software development.

Prerequisites

- Java basics
- Programming fundamentals
- Data structures

■ Learning Outcomes

ID	Outcome	Bloom's Level
LO1	Understand Java syntax and semantics	Remember
LO2	Apply object-oriented programming principles in Java	Understand
LO3	Design and implement Java programs using data structures	Apply
LO4	Analyze Java programs for efficiency and effectiveness	Analyze
LO5	Evaluate Java programs for quality and reliability	Evaluate
LO6	Create complex Java applications using advanced concepts	Create

Semester 1

Java Fundamentals and Data Structures

Focus: This semester focuses on Java syntax, object-oriented programming, and data structures.

Outcomes: LO1 LO2 LO3

Weekly Study Plan

Wk	Topic	Subtopics	Method	Hrs	Assessment
1	Introduction to Java	Java History, Java Features, Setting up Java Environment	Lecture	3h	—
2	Java Syntax	Variables, Data Types, Operators	Lab	3h	Lab
3	Control Structures	If-Else Statements, Switch Statements, Loops	Lecture	3h	Quiz
4	Functions and Methods	Method Declaration, Method Calling, Method Overloading	Lab	3h	Assignment
5	Object-Oriented Programming	Classes, Objects, Constructors	Lecture	3h	Lab
6	Inheritance and Polymorphism	Inheritance, Polymorphism, Method Overriding	Lab	3h	—
7	Exception Handling	Try-Catch Blocks, Throwing Exceptions, Exception Types	Lecture	3h	Quiz
8	Arrays and Lists	Arrays, Lists, Array Lists	Lab	3h	—
9	Data Structures	Stacks, Queues, Linked Lists	Lecture	3h	Lab
10	Trees and Graphs	Binary Trees, Graphs, Traversal Algorithms	Lab	3h	Assignment
11	Hash Tables and Maps	Hash Tables, Maps, Set Operations	Lecture	3h	—
12	File Input/Output and Persistence	File Input/Output, Serialization, Deserialization	Lab	3h	Quiz
13	Java Collections Framework	Collection Interfaces, Collection Classes, Iterator	Lecture	3h	Lab
14	Java Stream API	Stream Operations, Intermediate Operations, Terminal Operations	Lab	3h	—
15	Project Development	Project Planning, Project Implementation, Project Testing	Project	3h	Assignment
16	Final Project Presentations	Project Presentations, Code Review, Feedback	Project	3h	Project

Assessments

Assignments

Assignment	Due Week	Marks
Java Syntax Assignment	Week 4	10m
Data Structures Assignment	Week 10	10m
Project Proposal	Week 15	10m

Quizzes

Quiz	Week	Marks
Java Syntax Quiz	Week 3	5m
Data Structures Quiz	Week 7	5m
Java Collections Framework Quiz	Week 12	5m

Project Summary and Technical Implementation

EduSync is a Generative AI-powered educational platform designed to automate curriculum design and provide personalized learning experiences. The system eliminates the time-consuming process of manual curriculum planning by utilizing the Groq API integrated with the Llama 3.3 70B Large Language Model (LLM) to generate structured, high-quality, and industry-relevant educational content.

The platform serves two primary users: educators and students. Educators can provide course details such as course name, academic level, number of semesters, weekly hours, credits, and specialization. Based on these inputs, the AI generates a complete curriculum including semester-wise syllabus, Bloom's Taxonomy-based learning outcomes, assignments, quizzes, examinations, mini projects, capstone projects, skill mapping, and recommended learning resources. Students can generate personalized study plans, receive AI-based resource recommendations, and practice AI-generated assessments.

Technically, EduSync follows a modern full-stack architecture. The frontend is developed using HTML5, CSS3, JavaScript, and Bootstrap to provide a responsive and interactive user interface. The backend is implemented using Python FastAPI, which handles API requests, business logic, user management, and communication with the AI model. JWT-based authentication is implemented to provide secure login and role-based access control.

The system integrates with the Groq API and Llama 3.3 70B model using prompt engineering techniques to generate intelligent educational content. SQLite is used as the database for storing user accounts, generated curricula, learning plans, and assessment details. Additionally, a PDF generation module allows users to download professionally formatted curriculum reports.

Overall, EduSync combines Generative AI, modern web technologies, secure authentication, database management, and AI-driven content generation to deliver an efficient, scalable, and user-friendly solution for intelligent curriculum design and personalized education.

Challenges and Optimizations

During the development of EduSync, several challenges were encountered in integrating Generative AI with curriculum design. One major challenge was ensuring that AI-generated content was accurate, structured, and aligned with educational standards. This was addressed by implementing optimized prompt engineering techniques, providing clear instructions to the Llama 3.3 70B model to generate consistent curricula, learning outcomes, assessments, and project suggestions.

Another challenge was managing the response time of AI-generated content through the Groq API. This was optimized by designing efficient API communication and streamlining request processing to provide faster responses and a smoother user experience.

Maintaining consistency across semester-wise syllabi, skill mapping, and assessments was also challenging. This was improved through structured input validation and well-designed

curriculum generation workflows. Security and user management were enhanced by implementing JWT-based authentication, ensuring secure access for educators and students.

Additionally, efficient storage and retrieval of generated data were achieved through optimized SQLite database design and CRUD operations. The frontend was optimized using HTML, CSS, JavaScript, and Bootstrap to provide a responsive and user-friendly interface, while the PDF generation module was optimized to produce well-formatted curriculum reports efficiently.

Overall, these optimizations enhanced the accuracy, performance, security, reliability, and user experience of the EduSync platform.

Learning Outcomes

Through the development of EduSync, valuable experience was gained in building a complete full-stack Generative AI-powered web application by integrating modern frontend and backend technologies. The project provided practical knowledge in integrating the Groq API with the Llama 3.3 70B Large Language Model (LLM) for intelligent curriculum generation, personalized learning plans, and AI-based educational content creation. It enhanced understanding of prompt engineering, FastAPI development, REST API implementation, and secure authentication using JWT. The project also improved skills in database management using SQLite, CRUD operations, and designing responsive user interfaces with HTML, CSS, JavaScript, and Bootstrap. Additionally, it provided experience in handling AI responses, generating professional PDF reports, and optimizing system performance, security, and user experience. Overall, EduSync strengthened expertise in combining Generative AI, web development, database systems, API integration, and software engineering practices to develop an intelligent and efficient educational platform.

Future Scope

EduSync can be further enhanced by incorporating more advanced AI capabilities and educational features to improve the overall learning experience. Future improvements may include integration with multiple AI models for more diverse and accurate curriculum generation, support for multilingual content generation, and the addition of advanced personalization techniques based on student learning patterns and performance. The platform can be expanded with real-time collaboration features for educators, integration with Learning Management Systems (LMS), and cloud-based database solutions to support larger user communities.

Furthermore, future versions can include AI-powered progress tracking, adaptive assessments, automated feedback systems, and enhanced analytics to provide deeper insights into student performance and curriculum effectiveness. The system can also be deployed as a scalable cloud-based application with improved security, mobile application support, and wider accessibility, making EduSync a more comprehensive and intelligent solution for the future of digital education.

Conclusion

EduSync successfully demonstrates the potential of Generative AI in transforming academic curriculum design and personalized learning. By integrating the Groq API with the Llama 3.3 70B Large Language Model, the platform automates the creation of structured curricula, learning outcomes, assessments, projects, and personalized study plans, significantly reducing the time and effort required in traditional curriculum development.

The implementation of a modern full-stack architecture using HTML, CSS, JavaScript, Bootstrap, FastAPI, SQLite, and JWT authentication ensures a secure, responsive, and efficient user experience. The system provides educators with intelligent curriculum generation tools while enabling students to access personalized learning pathways and AI-generated assessments.

Overall, EduSync combines Generative AI, web technologies, and educational methodologies to deliver a smart, scalable, and user-friendly solution for modern education. The project highlights the ability of AI-driven systems to improve curriculum quality, enhance learning experiences, and support the future evolution of intelligent educational platforms.

Conclusion

GenAI Curriculum Generator successfully delivers an intelligent curriculum design assistant that combines advanced AI with educational expertise. By leveraging local Ollama deployment with Granite 3.3 2B, the system provides fast, accurate, and personalized curriculum recommendations without requiring cloud API keys. The platform demonstrates the viability of AI-powered curriculum design, making professional educational planning accessible to educators, institutions, and course designers while maintaining high performance, accuracy, and data privacy through local processing.